Dudas — Etapa 3 (chequeo de declaraciones)

Cada punto trae un ejemplo, la salida que da hoy el compilador y por qué. Para contestar alcanza con confirmar o con elegir la alternativa. Las líneas de los ejemplos cuentan desde 1.

Criterio general

Usted indicó que los logros son independientes entre etapas. Por eso el semántico se implementa para el MiniJava base del documento «Semántica de MiniJava – Parte I». Lo que agregan los logros de la Etapa 2 (`private`, genéricos anidados, atributos inicializados) se sigue aceptando sintácticamente, pero no suma controles en esta etapa.

¿Es correcto?

A. Etapa 3

1. ¿Hay que exigir un `main`?

`class Aa { }` → `[SinErrores]`.

El documento no lo pide, y Java compila sin `main`. La duda aparece porque los 5 casos correctos del tester traen `class Init { static void main() { } }`. Si hay que exigirlo, falta saber dos cosas: si alcanza con que haya uno, y qué lexema y qué línea se reportan, porque no hay ningún token al que apuntar.

2. `private` y herencia.

```
class Aa { private int x; }
class Bb extends Aa { int x; }
```

→ `[Error:x|2]`: el atributo ya está declarado en un ancestro.

Por el criterio general, la visibilidad no afecta ningún control. En Java este programa compila, porque un miembro `private` no se hereda.

3. Genéricos anidados.

```
class Caja<T> { }
class Par<T> { }
class Aa { Caja<Par<Nada>> x; }
```

→ `[SinErrores]`, aunque `Nada` no existe.

Se controla sólo el primer nivel: que `Caja` sea genérica y que `Par` exista. El documento define el argumento como «el nombre de una clase o interfaz existente». Controlar el anidamiento es el logro Genericidad Avanzada E3, que no perseguimos.

4. Herencia circular: ¿qué token se reporta?

```
class Aa extends Bb { }
class Bb extends Aa { }
```

→ `[Error:Aa|1]`: la primera clase declarada que forma parte del ciclo.

Para lo que no tiene un token claramente asociado, la Pautas slide 30 pide usar el nombre de la entidad con el problema. `javac` marca el mismo lugar: «cyclic inheritance involving Aa».

Alternativa: `[Error:Bb|1]`, el tipo que aparece en el `extends`.

5. Argumento genérico sobre un tipo que no es genérico: ¿qué token se reporta?

```
class Caja { }
class Aa { Caja<String> x; }
```

→ `[Error:Caja|2]`: el tipo que no declara parámetro. `javac` dice «type Caja does not take parameters».

Alternativa: `[Error:String|2]`, el argumento.

6. `extends String` y `extends System`.

`class Aa extends String { }` → `[Error:String|1]`.

Es como en Java, donde las dos clases son `final`: `javac` dice «cannot inherit from final String». El documento sólo dice que las dos «heredan de Object».

7. El parámetro de tipo en un método estático.

`class Caja<T> { static T crear() { } }` → `[Error:T|1]`.

Es como en Java: `javac` dice «non-static type variable T cannot be referenced from a static context». El documento permite usar `T` «en atributos, retornos y parámetros formales», y no distingue los métodos estáticos.

8. Una interfaz no puede contradecir a `Object`.

`interface Ix { int toString(); }` → `[Error:toString|1]`.

Es como en Java, donde una interfaz incluye los métodos públicos de `Object`: `javac` dice «toString() in Ix cannot override toString() in Object».

9. Un método estático que coincide con uno de la interfaz: ¿qué token se reporta?

```
interface Ix { void m(); }
class Aa implements Ix { static void m() { } }
```

Que es error está fuera de duda: el documento §6.2 dice que un método estático no implementa uno de interfaz.

→ `[Error:m|2]`: el método que está mal. `javac` marca el mismo lugar: «m() in Aa cannot implement m() in Ix».

Alternativa: `[Error:Aa|2]`, la clase que no implementa.

10. En esta etapa los cuerpos no se chequean.

`class Aa { void m() { this.b().a = 5; } }` → `[SinErrores]`, aunque `b` no está declarado.

El documento §1 deja los cuerpos para la Etapa 4. Es el mismo caso que `sintCorrecto04`, del tester de la Etapa 2.

B. Para la Etapa 4 (lo que la Etapa 2 dejó al semántico)

11. `int x, y, z = 10;` (logro Variables Locales Clásicas).

En Java esta línea inicializa **sólo** `z`: si después se usa `x`, `javac` dice «variable x might not have been initialized». ¿Se lee igual, o las tres variables valen 10?

12. Lo que el sintáctico deja pasar a propósito.

Todos estos casos pasan hoy el análisis sintáctico. ¿Se controlan todos en la Etapa 4, como en Java?

Ejemplo	Qué se controla
`5 = x;`	que el lado izquierdo del `=` sea una variable
`5 + 3;`	que la sentencia sea una asignación o una llamada (Java: «not a statement»)
`m()++;`	que el operando de `++` o `--` sea una variable
`noDeclarada = 5;`	que los nombres estén declarados
`int x = "hola";` y `return;` en un método `int`	la compatibilidad de tipos
`m(1, 2, 3)` con `m` de un solo parámetro	la aridad de la llamada
`int x; int x;` en un mismo bloque	las variables locales repetidas
`new int[]{{}}`	que el inicializador coincida con las dimensiones
`3.5e38f`	el rango del `float` (Java: «floating-point number too large»)
`a + b * c`	la precedencia: la gramática de la cátedra pone los 13 operadores binarios en un solo nivel. ¿Se aplica la de Java?

C. Diferencias con los casos de prueba de un compañero

Corrimos los 93 casos de prueba de un compañero y coincidimos en 86. En los 7 restantes, los dos compiladores detectan el mismo error en la misma línea: sólo cambia el lexema reportado. Son exactamente las consultas 4 y 5.

Herencia circular (3 casos).

```
class ClaseUno extends ClaseDos { }
class ClaseDos extends ClaseTres { }
class ClaseTres extends ClaseUno { }
```

→ Él reporta `[Error:ClaseDos|1]`, el tipo del `extends` de la primera clase. Nosotros, `[Error:ClaseUno|1]`, la clase.

Por qué: la Pautas slide 30 pide el nombre de la entidad con el problema cuando no hay un token claramente asociado, y `javac` marca la clase: «cyclic inheritance involving ClaseUno», con el cursor en `class ClaseUno`. Lo mismo pasa con un ciclo de interfaces y con uno de cuatro clases.

Argumento genérico sobre un tipo que no es genérico (4 casos).

```
class Simple { }
class Derivada extends Simple<String> { }
```

→ Él reporta `[Error:String|2]`, el argumento. Nosotros, `[Error:Simple|2]`, el tipo.

Por qué: `String` existe y sería un argumento válido; lo que está mal es que `Simple` no declara un parámetro de tipo. `javac` responsabiliza al tipo: «type Simple does not take parameters». Lo mismo pasa con `implements`, con un atributo y con un parámetro.